

ReduLink: Authenticated Reference Substitution for Redundancy-Suppressed Transfers over Encrypted QUIC Streams

Michél Nguyen

University of the People | minh.systems | ORCID: 0000-0001-6834-4422

Abstract

ReduLink is an endpoint-controlled representation layer for encrypted WAN transfers. It replaces repeated payload chunks with compact references authenticated against epoch, scope, stream identifier, reconstructed offset, length, nonce, and chunk identity. The contribution is not a new matching algorithm and not a claim of faster physical links. The contribution is authenticated, scoped, QUIC-compatible reference substitution: a way to use redundancy in warm endpoint state while preserving explicit reconstruction, privacy, and fail-closed semantics.

The artifact maps compact binary ReduLink messages into TLS-protected aioquic QUIC streams, implements semantic MISS/FULL repair, uses an exporter-style key schedule model, and tests replay and tamper rejection. Evaluation includes fixed-block reuse, gzip/zstd, real rsync, negative public source-release pairs, object-aligned public source-release transfers, a public Redis-derived layer-like case, repeated native QUIC trials, block-size sensitivity, component-cost measurements, and conservative fairness accounting. Results show that fixed reuse or rsync often wins for file-tree synchronization, while ReduLink is useful for object-aligned or layer-like warm-state transfers where authenticated stream-compatible references are the desired abstraction.

Quantitatively, on object-aligned public release transfers ReduLink reaches 1.94x to 9.70x effective stream-payload reduction (and up to 21.37x on a layer-like positive case), within a few percent of an idealized fixed-object-reuse baseline, while the authenticated variant stays within a few additional percent; on ordinary source-tree updates rsync wins by up to 73x, which we report rather than hide. Native aioquic stream experiments confirm byte-exact reconstruction with deterministic stream multipliers under zero and periodic datagram loss.

Keywords: QUIC; redundancy elimination; data deduplication; content-defined chunking; authenticated references; encrypted transport; reproducible evaluation.

1. Introduction

Encrypted transports restrict transparent in-network redundancy elimination. QUIC combines UDP transport, TLS security, stream multiplexing, loss recovery, and congestion control, which makes middlebox rewriting of payloads the wrong abstraction for most public or end-to-end encrypted deployments [6-10].

The practical question is therefore not whether repeated bytes can be replaced by references. Fixed-block reuse, rsync, and low-bandwidth file systems already demonstrate that repeated bytes can be avoided [1-5]. The question is whether a reference-substitution mechanism can be made explicit, authenticated, scoped, and compatible with encrypted endpoint streams without claiming a universal accelerator.

Effective multiplier means reconstructed or input bytes divided by encoded bytes at the stated accounting layer. Stream-payload multipliers exclude UDP/IP/link overhead. UDP-estimated multipliers add a local IPv4/UDP estimate. Congestion fairness, where claimed, applies to encoded bytes, not reconstructed bytes.

This paper investigates three questions. RQ1: can reference substitution over encrypted QUIC streams be made explicit, authenticated, and privacy-scoped without custom transport-layer changes? RQ2: under which workload shapes does authenticated reference substitution save bytes, and where do simpler fixed-block reuse, compression, or rsync win? RQ3: what is the component-level cost of the authentication and accounting machinery, and how stable are the native QUIC results across repeated runs?

2. Contributions and Claim Boundary

- An authenticated ReduLink reference format and validation model for endpoint-controlled encrypted streams.

- A compact binary mapping carried inside native aioquic QUIC streams, with byte-exact semantic repair and repeated trials.
- An exporter-style artifact key schedule and security tests for wrong secret, scope, epoch, stream id, offset, length, replay, and tampering.
- A comparison against fixed-block reuse, gzip/zstd, and real rsync, including cases where these baselines win decisively.
- External public source-release negative evidence, object-aligned public source-release workloads, and a public Redis-derived layer-like positive case.
- Block-size sensitivity, component-cost measurements, and conservative accounting-layer separation.

The central claim is conditional: ReduLink helps when byte-stable chunks recur across warm endpoint state and authenticated reference substitution is preferable to a file-oriented delta protocol or local compression. It is not a replacement for rsync or compression, and the artifact is a native QUIC stream mapping rather than custom QUIC extension frames.

A practical example is a registry, CDN, backup, or update service that repeatedly sends object records to the same endpoint or tenant while preserving end-to-end QUIC encryption. In that setting, the server can reference receiver-known objects or chunks without exposing plaintext to a middlebox and without asking the application to become a file-tree synchronization protocol. The price paid by ReduLink is authentication and metadata overhead; the benefit is explicit context binding, safe miss repair, and a stream-compatible representation.

3. Related Work and Motivation

Spring and Wetherall introduced protocol-independent redundancy elimination, and later work studied enterprise redundancy, coordinated redundancy elimination, and endpoint redundancy elimination [1-4]. LBFS and rsync-family systems show the value of low-bandwidth file synchronization when both sides operate over file objects [5]. Content-defined chunking and chunking security have a substantial literature [11-13]. Deduplication side channels motivate careful dictionary scoping and avoidance of global cross-user private dictionaries [16,17].

ReduLink is motivated by a narrower deployment class: encrypted object delivery over QUIC streams, registry or CDN transfers where the receiver has prior same-origin state, backup-like page streams, and administrative domains where per-origin or per-tenant dictionaries are explicit policy. Modern Ethernet line rates continue to increase [14], so the paper avoids physical-rate claims and focuses only on representation-layer savings.

The system boundary is deliberately endpoint-side. Middlebox WAN optimizers can exploit redundancy only when they see or terminate plaintext. ReduLink instead assumes the cooperating endpoints already control the plaintext before QUIC protection and can decide whether a reference is safe. This difference matters for HTTP/3, CDN edge delivery, update agents, enterprise VPN clients, and backup clients, where the transport can remain encrypted while the application or endpoint stack performs representation substitution.

3.1 Deployment Model

The deployment model has three roles: a sender with access to the new byte stream, a receiver with a scoped warm dictionary, and a policy domain that defines whether dictionary state is per connection, per origin, or per tenant. Public Internet mode defaults to per-connection dictionaries. Public artifact mode may use signed or manifest-controlled per-origin dictionaries. Enterprise mode may use per-tenant dictionaries with audit and quota controls. Global private cross-user dictionaries remain out of scope.

Table 1. ReduLink deployment modes and dictionary scoping.

Deployment	Dictionary scope	Why ReduLink rather than rsync/compression
CDN or registry objects	Per origin or per client	Objects arrive as encrypted streams; file-tree delta negotiation may not be the serving abstraction.

Backup/page streams	Per endpoint or tenant	Page-like chunks recur across snapshots and can be referenced while preserving authenticated reconstruction.
Enterprise VPN/admin domain	Per tenant	Policy can allow shared warm state while keeping middleboxes away from plaintext.
Source-tree sync	File tree	Usually better served by rsync; ReduLink is not targeted here.

4. Protocol and Security Model

The sender chunks outgoing bytes and emits FULL frames for new chunks or REF frames for chunks already believed present at the receiver. A REF is deliverable only if dictionary membership, epoch, scope, stream id, reconstructed offset, length, nonce, chunk identity, and authentication tag validate. Otherwise the receiver fails closed or requests semantic FULL repair.

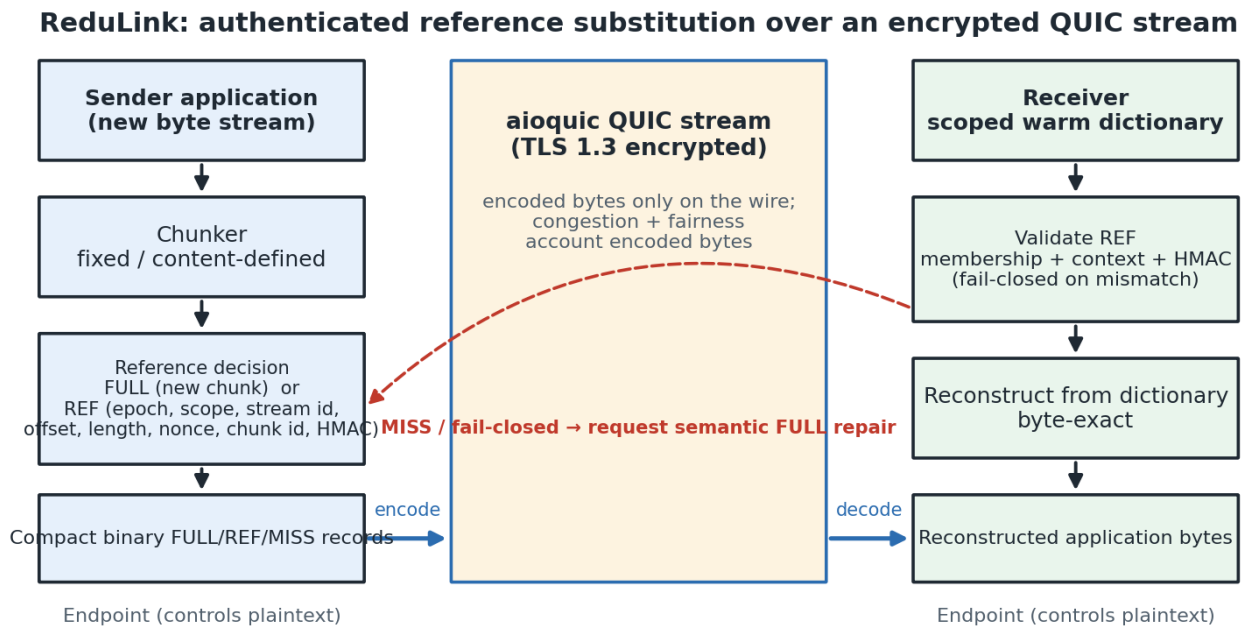


Figure 1. ReduLink protocol and deployment architecture. The sender chunks outgoing bytes and emits FULL or authenticated REF records carried as compact binary application data inside an encrypted QUIC stream; the receiver validates each reference against dictionary membership and bound context before byte-exact reconstruction, failing closed and requesting semantic FULL repair on any mismatch.

The default privacy scope is per connection. Shared per-origin dictionaries are appropriate only for public artifacts or same-tenant administrative domains. The design excludes global cross-user private dictionaries. The artifact binds frames with an HMAC-based model and an exporter-style key schedule; production QUIC integration should use QUIC TLS exporter material and transport-parameter negotiation.

Table 2. Threat model: ReduLink controls and corresponding artifact evidence.

Threat	ReduLink control	Artifact evidence
Reference forgery	HMAC binds epoch, scope, stream, offset, length, nonce, and chunk identity.	Tamper and wrong-context tests.
Replay	Nonce and context binding reject reused frames.	Replay rejection tests.
Dictionary poisoning	Receiver admits only authenticated FULL data or policy-controlled warm state.	Dictionary membership and reconstruction tests.
Expansion abuse	References expand only to negotiated chunk lengths and dictionary entries.	Accounting and reconstruction checks.
Content-existence leakage	No global private cross-user dictionary; scope is explicit policy.	Threat model and mode table.

5. Implementation and Metrics

The artifact is implemented in Python for reproducibility and uses aioquic for native QUIC stream experiments [18]. ReduLink messages are compact binary application-stream records inside QUIC streams. This exercises real QUIC handshake and encryption, but it does not implement custom QUIC frame types or transport parameters.

Three byte layers are reported. Input bytes are the reconstructed application bytes. Stream-payload bytes are the ReduLink or raw application bytes written into QUIC streams. UDP-estimated bytes add a local IPv4/UDP estimate from the loopback proxy path. The paper avoids treating stream-payload multipliers as full wire-rate measurements; they isolate the representation layer so that raw QUIC, ReduLink, gzip, zstd, fixed reuse, and rsync-style baselines can be compared consistently.

The compact binary format is included because the JSON baseline was intentionally readable but inflated. The binary format encodes frame type, stream id, reconstructed offset, length, chunk id, nonce, authentication tag, and payload only where required. It is still application-stream data, not QUIC extension-frame data.

Table 3. Evidence hierarchy: what each layer of evidence supports and does not prove.

Evidence	Supports	Does not prove
Model	FULL/REF accounting and reconstruction	Transport behavior
Secure model	HMAC binding and replay checks	Live QUIC exporter use
aioquic stream	Encrypted QUIC stream mapping	Custom extension frames
UDP/IPv4 estimate	Local datagram accounting	Full packet capture
Workloads	Workload sensitivity	Universal acceleration

6. Evaluation Methodology

Evaluation uses four classes of evidence. First, deterministic journal fixtures isolate predicted positive and negative workload shapes. Second, public source-release pairs test ordinary source-tree updates where ReduLink should not be assumed to help. Third, object-aligned public release experiments use the same public bytes but model registry/CDN object delivery rather than tarball synchronization. Fourth, aioquic stream experiments measure the native encrypted stream mapping on positive and negative cases.

For each workload, the paper reports ReduLink fixed chunking, ReduLink content-defined chunking where relevant, fixed-block reuse, compression, and rsync where the baseline is structurally applicable. The fixed-block baseline is intentionally strong and simple. If it beats ReduLink, the result is reported rather than hidden, because ReduLink is a security and transport-compatibility layer over reuse, not a superior matching algorithm.

7. Workload Results and Baseline Interpretation

Table 4. Warm-state workload results (effective stream-payload multiplier): ReduLink versus fixed-block reuse and best compression.

Workload	ReduLink	Fixed reuse	Best compression	RL - fixed
disk snap	28.49x	31.97x	1.06x	-3.48x
oci layer	11.05x	10.68x	1.50x	+0.37x
package meta	0.99x	9.88x	14.31x	-8.89x
repository snap	24.73x	24.60x	10.37x	+0.14x
logs	0.99x	14.89x	18.01x	-13.89x
compressed neg	0.99x	1.00x	1.00x	-0.01x

The table is the interpretation guard. On most byte-stable workloads, fixed-block reuse matches or beats ReduLink. The distinct ReduLink contribution is not superior matching; it is authenticated, scoped, stream-compatible reference substitution with explicit repair and privacy rules.

7.1 External Public Source Releases

Table 5. External public source-release pairs: ReduLink and fixed reuse versus real rsync (negative case for ReduLink).

Public pair	ReduLink	Fixed reuse	rsync total
click-8.1.7 to 8.1.8	0.99x	0.99x	6.48x
redis-7.2.4 to 7.2.5	1.00x	1.00x	73.13x
nginx-1.25.3 to 1.25.4	1.00x	0.99x	49.49x

The independent source-release pairs are negative for ReduLink and strongly positive for rsync. This is an important result. Ordinary related source trees are better served by file-oriented delta transfer than by stream-level reference substitution.

7.2 Object-Aligned Public Release Transfers

Table 6. Object-aligned public release transfers modeling registry/CDN object delivery.

Public release	File stability	Bytes	ReduLink	Secure RL	Fixed object reuse	gzip
click	80/144 unchanged	947,826	1.94x	1.91x	1.94x	2.86x
redis	1557/1582 unchanged	15,467,095	9.70x	9.04x	9.68x	4.61x
nginx	468/503 unchanged	7,357,392	4.38x	4.25x	4.38x	6.09x

This experiment uses the same public release tarballs but changes the transfer abstraction. Files are treated as object records and chunked independently, modeling registry/CDN object delivery rather than raw tarball or source-tree synchronization. Redis and nginx show positive ReduLink results because many public release objects remain byte-identical; Click is weaker because fewer files are unchanged. This is a stronger external positive signal than a purely synthetic corpus, but it is still not a captured production trace.

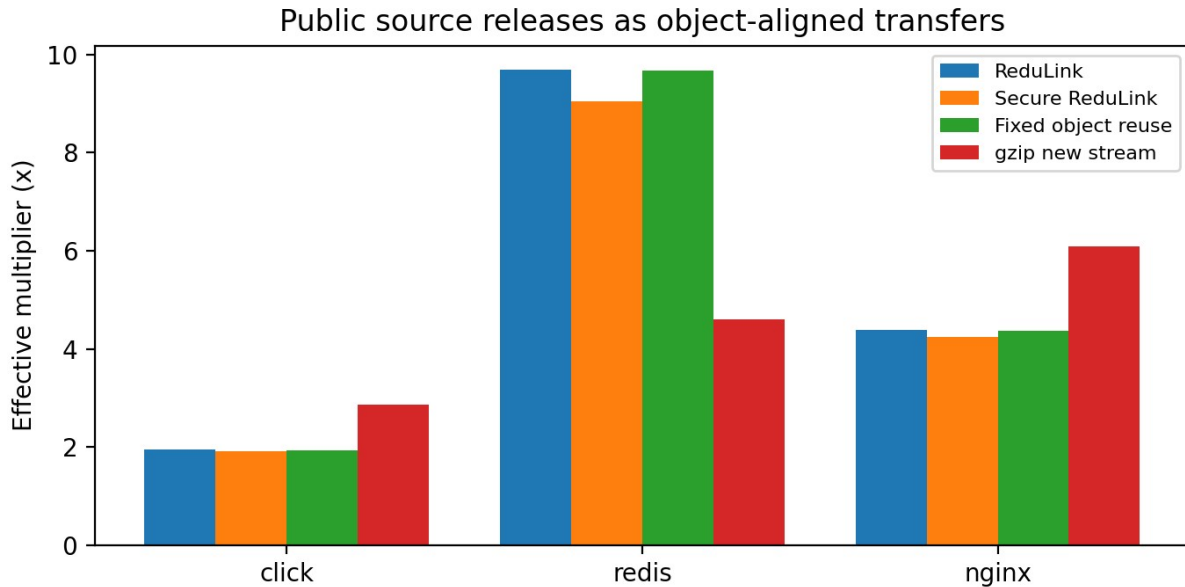


Figure 2. Object-aligned public release transfers: effective stream-payload multipliers for ReduLink, the authenticated variant, fixed object reuse, and gzip.

7.3 Layer-Like External-Positive Case

Table 7. Layer-like external-positive case derived from public Redis release bytes.

Case	Input bytes	ReduLink	Secure RL	Fixed reuse
redis-layered-public-positive	524,288	21.37x	18.96x	21.33x

The Redis-layered case is derived from included public Redis release bytes and aligned changed blocks. It is not a production trace; its role is to test the predicted positive case for layer-like objects where stable chunks remain aligned.

7.4 Block-Size Sensitivity

Table 8. Block-size sensitivity of the effective multiplier (fixed chunking).

Workload	1 KiB	2 KiB	4 KiB	8 KiB	16 KiB
disk snapshot	17.13x	23.33x	28.49x	17.08x	8.99x
oci layer	12.11x	11.86x	11.05x	5.91x	3.60x
repository snapshot	20.21x	24.99x	24.73x	14.45x	7.55x

The best block size is workload-dependent. The artifact therefore treats 4 KiB as a reproducible default, not as a universal optimum.

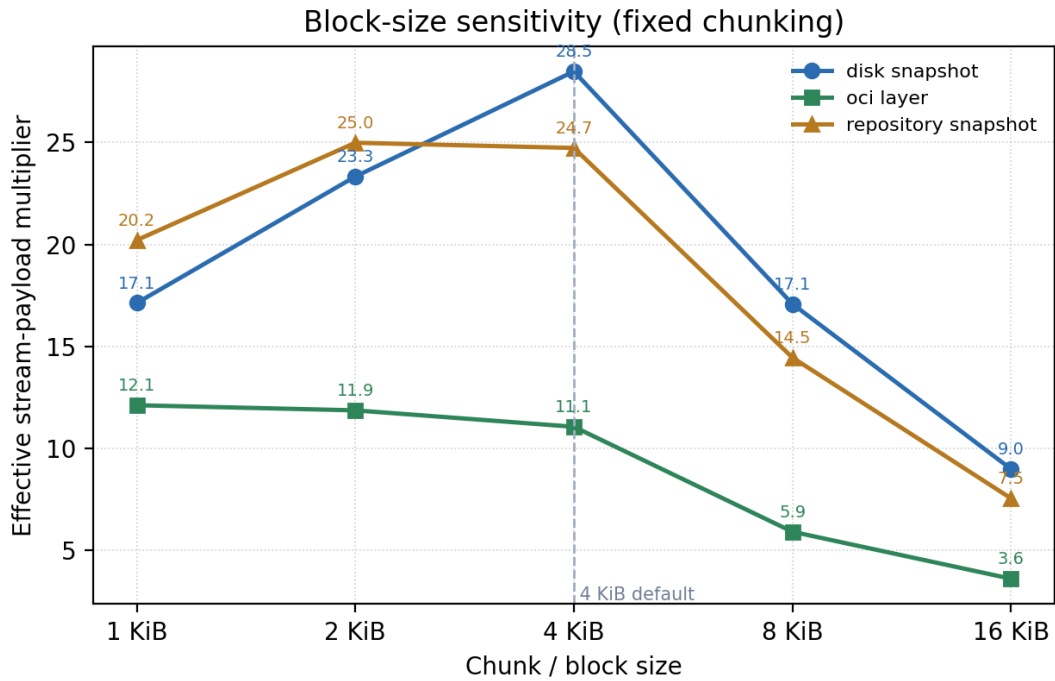


Figure 4. Block-size sensitivity of the effective multiplier for fixed chunking across three warm-update fixtures.

8. Native QUIC Experiments

Table 9. Native aioquic stream mapping: raw versus ReduLink under zero and periodic datagram loss.

Method	Loss every	Stream x	UDP-est x	Recon.
Raw	0	1.00x	0.90x	OK
ReduLink	0	3.19x	2.60x	OK
Raw	9	1.00x	0.78x	OK
ReduLink	9	3.19x	2.38x	OK

Table 10. Native aioquic stream mapping across positive and negative workload cases.

Case	Input	Stream x	UDP-est x	Misses	Recon.
demo positive	98,304	3.19x	2.57x	13	OK
compressed negative	262,242	0.91x	0.84x	0	OK
Redis layered	524,288	3.83x	3.46x	72	OK

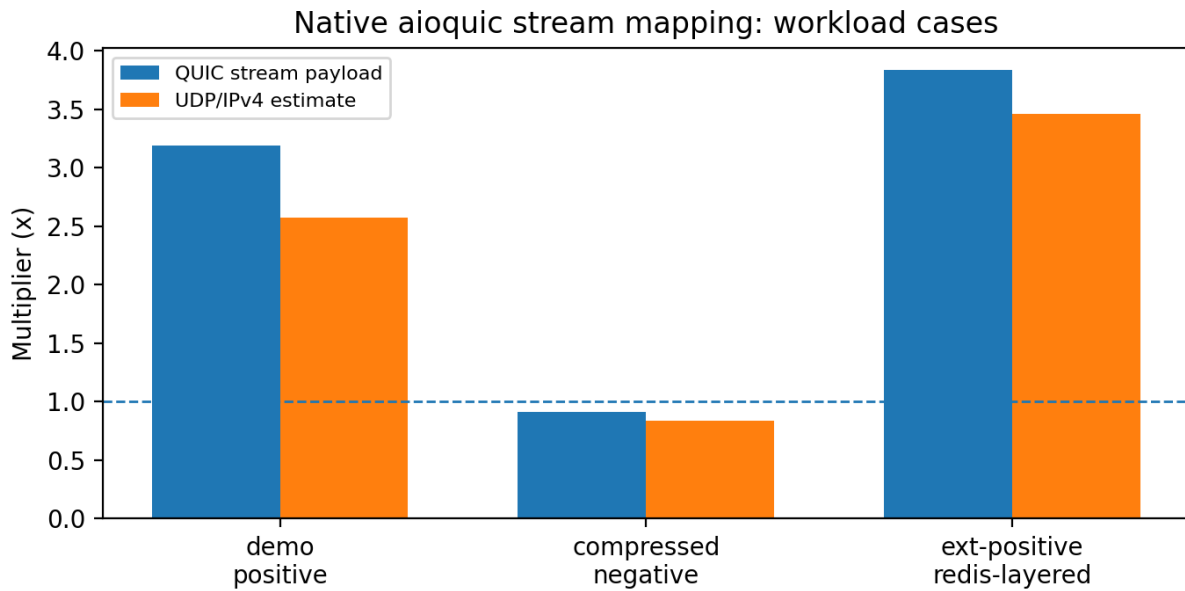


Figure 3. Native aioquic stream mapping on positive and negative workload cases.

Table 11. Repeated native QUIC trials (n = 3): run-to-run stability of multipliers and elapsed time.

Metric	n	Mean	Std. dev.	Min	Max
raw client ms	3	227.043667	19.900327	205.943000	245.474000
redulink stream multiplier	3	3.189928	0.000000	3.189928	3.189928
redulink udp est multiplier	3	2.578853	0.003857	2.574954	2.582666
redulink client ms	3	137.076000	4.472720	131.991000	140.401000

The repeated-trial table is intentionally small but avoids reporting only one favorable QUIC run. The multiplier is deterministic for fixed bytes, while elapsed times vary across runs.

9. Component Costs

Table 12. Component-level cost measurements (local Python artifact timing; not production line-rate).

Component	Input bytes	MiB/s local	Roundtrip
fixed chunking	2,097,152	6237.0	not applicable
CDC chunking	2,097,152	7.5	not applicable
model fixed roundtrip	1,573,964	680.0	OK
HMAC roundtrip	1,573,964	192.2	OK
wire encode	1,573,964	4696.9	not applicable
wire decode	1,573,964	2451.2	not applicable

The Python CDC implementation is slow and is not production line-rate evidence. Fixed chunking, HMAC validation, and compact binary encoding are much faster in the local artifact. A production implementation would require optimized native chunking and tighter integration with the transport stack.

10. Fairness and Accounting

ReduLink must not receive congestion credit for reconstructed bytes. Congestion control and bottleneck accounting count encoded bytes. The package includes wire-byte accounting, QUIC smoke experiments, local UDP/IPv4 byte estimates, and bottleneck emulation. It does not include a full tc/netem or Mininet congestion-control study.

Table 13. Fairness and accounting summary.

Metric	Value
Fairness rule	Congestion counts encoded bytes, not reconstructed bytes
Live congestion-control study	Not included
Accounting layer	Stream payload and local UDP/IPv4 estimate

11. Why Not Just Compress or Use rsync?

Compression is best when repetition is local to one object. rsync is best when both endpoints share a file tree or file object and can run a delta protocol. The ReduLink use case is different: encrypted endpoint streams where dictionary state is already available at the receiver, but the transfer is not naturally a file-tree synchronization session.

The evaluation confirms the boundary. On source-release trees, rsync dominates. On package metadata and logs, compression or fixed-block reuse dominates. On object-aligned public releases and layer-like byte-stable objects, ReduLink provides authenticated reference substitution at modest overhead over a simple fixed-reference baseline. The practical motivation is security/context binding and stream compatibility, not beating rsync in rsync-shaped deployments.

The motivating cases are therefore not arbitrary file synchronization. They are stream-serving cases in which the endpoint already knows that a receiver has a dictionary state but must still authenticate every reconstruction step and preserve encrypted transport semantics. A CDN edge, registry, backup client, or enterprise service may choose this representation because it avoids exposing plaintext to a WAN optimizer and avoids changing the application protocol into a file-tree delta session.

12. Limitations and Future Work

The implementation maps ReduLink into QUIC streams rather than adding custom QUIC extension frames or transport parameters. It uses an exporter-style key schedule model rather than live QUIC TLS exporter bytes. The public object-sequence workload is derived from real release bytes, but it is still a transfer-model experiment rather than a captured production trace. The fairness evidence remains conservative and does not replace a full network-emulator congestion study.

Future work should add custom QUIC frame integration, live exporter-derived keys, tc/netem or Mininet fairness experiments, packet capture, and independently captured registry or backup traces.

13. Reproducibility

The package contains source code, public corpora, generated corpora, benchmark scripts, result CSV/JSON files, figures, a citation checker, and tests [15]. The reviewer-facing smoke command is: `python3 scripts/run_smoke_validation.py`. Full validation is: `python3 scripts/run_full_validation.py`. Aioquic tests skip gracefully when aioquic is unavailable; installing `requirements-dev.txt` enables complete QUIC stream validation.

14. Conclusion

ReduLink is best understood as authenticated, scoped reference substitution for encrypted endpoint streams. Its byte savings are conditional and often reproduced or exceeded by simpler fixed-block reuse, compression, or rsync. The contribution is to make reference substitution explicit, authenticated, privacy-scoped, repairable, and compatible with native QUIC stream transport. The evidence supports ReduLink for warm-state object-aligned or layer-like transfers, while negative source-release results show where it should not be used.

Data and Code Availability

All source code, public-corpus manifests, generated corpora, benchmark scripts, result CSV/JSON files, figures, the citation checker, and the test suite are openly available in the ReduLink repository [15] under the MIT License. Every

figure and table in this paper is regenerated from the committed result files by the build and validation scripts, so the reported numbers can be reproduced from the artifact. Large external public-release corpora are reconstructed from documented public sources rather than redistributed.

References

- [1] N. T. Spring and D. Wetherall, 'A protocol-independent technique for eliminating redundant network traffic,' ACM SIGCOMM, 2000.
- [2] A. Anand, V. Sekar, and A. Akella, 'SmartRE: An architecture for coordinated network-wide redundancy elimination,' ACM SIGCOMM, 2009.
- [3] A. Anand et al., 'Redundancy in network traffic: Findings and implications,' ACM SIGMETRICS, 2009.
- [4] B. Aggarwal et al., 'EndRE: An end-system redundancy elimination service for enterprises,' USENIX NSDI, 2010.
- [5] A. Muthitacharoen, B. Chen, and D. Mazieres, 'A low-bandwidth network file system,' ACM SOSP, 2001.
- [6] J. Iyengar and M. Thomson, 'QUIC: A UDP-based multiplexed and secure transport,' RFC 9000, IETF, 2021.
- [7] M. Thomson and S. Turner, 'Using TLS to secure QUIC,' RFC 9001, IETF, 2021.
- [8] J. Iyengar and I. Swett, 'QUIC loss detection and congestion control,' RFC 9002, IETF, 2021.
- [9] M. Kuehlewind and B. Trammell, 'Applicability of the QUIC transport protocol,' RFC 9308, IETF, 2022.
- [10] B. Trammell et al., 'Manageability of the QUIC transport protocol,' RFC 9312, IETF, 2022.
- [11] W. Xia, X. Zou, Y. Zhou, H. Jiang, C. Liu, D. Feng, Y. Hua, Y. Hu, and Y. Zhang, 'The design of fast content-defined chunking for data deduplication based storage systems,' IEEE Transactions on Parallel and Distributed Systems, vol. 31, no. 9, 2020.
- [12] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, 'A thorough investigation of content-defined chunking algorithms for data deduplication,' arXiv:2409.06066, 2024.
- [13] B. Alexeev, C. Percival, and Y. X. Zhang, 'Chunking attacks on file backup services using content-defined chunking,' arXiv:2504.02095, 2025.
- [14] IEEE 802.3 Ethernet Working Group, 'IEEE 802.3 Ethernet Working Group active projects,' accessed June 2026.
- [15] M. Nguyen, 'ReduLink artifact and reproducibility package,' GitHub repository, 2026. <https://github.com/pinkysworld/redulink-deduplex-quic>
- [16] D. Harnik, B. Pinkas, and A. Shulman-Peleg, 'Side channels in cloud services: Deduplication in cloud storage,' IEEE Security and Privacy, 2010.
- [17] M. Bellare, S. Keelveedhi, and T. Ristenpart, 'DupLESS: Server-aided encryption for deduplicated storage,' USENIX Security, 2013.
- [18] aioquic project contributors, 'aioquic: QUIC and HTTP/3 implementation in Python,' software repository, version 1.3.0, 2026. <https://github.com/aiohttp/aioquic>